

Design de Software II

PROF. SAULO R. BENATTI

Listas e Tuplas

Aula 2

- Hoje, vamos aprender sobre listas e tuplas, o que são essas coleções de elementos e como podemos aplicá-las em nossos projetos em Python.
- Vamos começar entendendo o que são as listas.
- As listas são coleções de elementos mutáveis. Isso significa que podemos ter uma série de itens que podem ser alterados após a criação dessa lista. Sua sintaxe é representada por colchetes.

- lista = [1, 2, 3]

- lista_mista = [1, "texto", 3.14, True, [1, 2, 3]]

- Teremos os elementos de uma lista entre colchetes e separados por vírgula. Um detalhe sobre as listas é que elas são flexíveis, ou seja, podem conter qualquer tipo de dados nos seus elementos.
- Podemos ter uma lista onde os elementos são do mesmo tipo, como uma lista com apenas valores numéricos, ou uma lista mista, onde misturamos valores numéricos, *strings*, booleanos e até outras listas.

Tuplas

- As tuplas são coleções de elementos imutáveis. Isso significa que, a partir do momento que criamos uma tupla, não podemos alterar seus elementos.
- A sintaxe é muito parecida com a das listas, mas, ao invés de utilizar colchetes, utilizamos parênteses. Elas também são flexíveis e podem conter vários tipos de dados.

- tupla = (1, 2, 3)
- tupla_mista = (1, "texto", 3.14, False, [1, 2, 3])
- Quando utilizamos essas coleções de elementos, queremos acessar seus itens. Temos uma série de operações comuns para isso.

Operações comuns: acesso por índice

- A primeira delas é o acesso por índice, que pode ser utilizado tanto em listas quanto em tuplas. Acessamos cada um dos elementos da nossa coleção através de um índice, que começa em zero e está dentro de colchetes.

```
# Acessa o elemento na posição 1 da lista, que é 20.
lista = [10, 20, 30]
print(lista[1])

# Acessa o elemento na posição 1 da tupla, que é 42.
tupla = (11, 42, 64)
print(tupla[1])
```

- No código, temos uma lista com três elementos. O primeiro elemento está no índice zero, o segundo no índice 1 e o terceiro no índice 2. Na sequência, temos um print, onde chamamos a nossa lista e, entre colchetes, passamos o índice 1.
- O print trará o elemento 20, que é o item nessa posição dentro da lista. O mesmo se aplica para as tuplas.

```
# Acessa o elemento na posição 1 da lista, que é 20.
lista = [10, 20, 30]
print(lista[1])

# Acessa o elemento na posição 1 da tupla, que é 42.
tupla = (11, 42, 64)
print(tupla[1])
```

Operações comuns: *slicing*

- A próxima operação comum é o *slicing*, que serve para extrair um pedaço da nossa lista ou tupla.
- Criamos uma subestrutura parecida com a dos índices, mas especificamos um intervalo de índices.

No código, temos uma lista com quatro elementos e chamamos um print com a nossa lista, fazendo o *slicing*, especificando que queremos extrair o pedaço que vai desde o índice 1 até o índice 3.

```
# Extrai os elementos do índice 1 até o 2 (não inclui o índice 3), resultando em [20, 30].
lista = [10, 20, 30, 40]
print(lista[1:3])

## Extrai os elementos do índice 1 até o 2 (não inclui o índice 3), resultando em [20, 30].
tupla = (10, 20, 30, 40)
print(tupla[1:3])
```

É importante ressaltar que esse índice final, [1:3], não está incluso no que vamos extrair dessa lista. Quando chamamos esse *slicing*, teremos de saída uma lista que começa com o elemento 20, que está na posição 1, o elemento 30, que está na posição 2, e, quando chega na posição 3, não inclui e finaliza a operação. O mesmo se aplica para as tuplas.

Operações comuns: operador in

- Na sequência, temos o operador in, que serve para verificar se um elemento pertence a uma lista ou tupla. Ele retorna TRUE se o elemento fizer parte da coleção, e FALSE caso contrário.

```
lista = [10, 20, 30]

print(20 in lista) # Verifica se 20 está presente na lista, retorna True
print(40 in lista) # Verifica se 40 está presente na lista, retorna False

tupla = (10, 20, 30)
print(20 in tupla) # Verifica se 20 está presente na tupla, retorna True
print(40 in tupla) # Verifica se 40 está presente na tupla, retorna False
```

- No código, temos uma lista e chamamos dentro do print essa verificação. Verificamos 20 em lista, se o 20 faz parte da lista, e o print exibirá TRUE, porque o 20 faz parte da lista.

```
lista = [10, 20, 30]

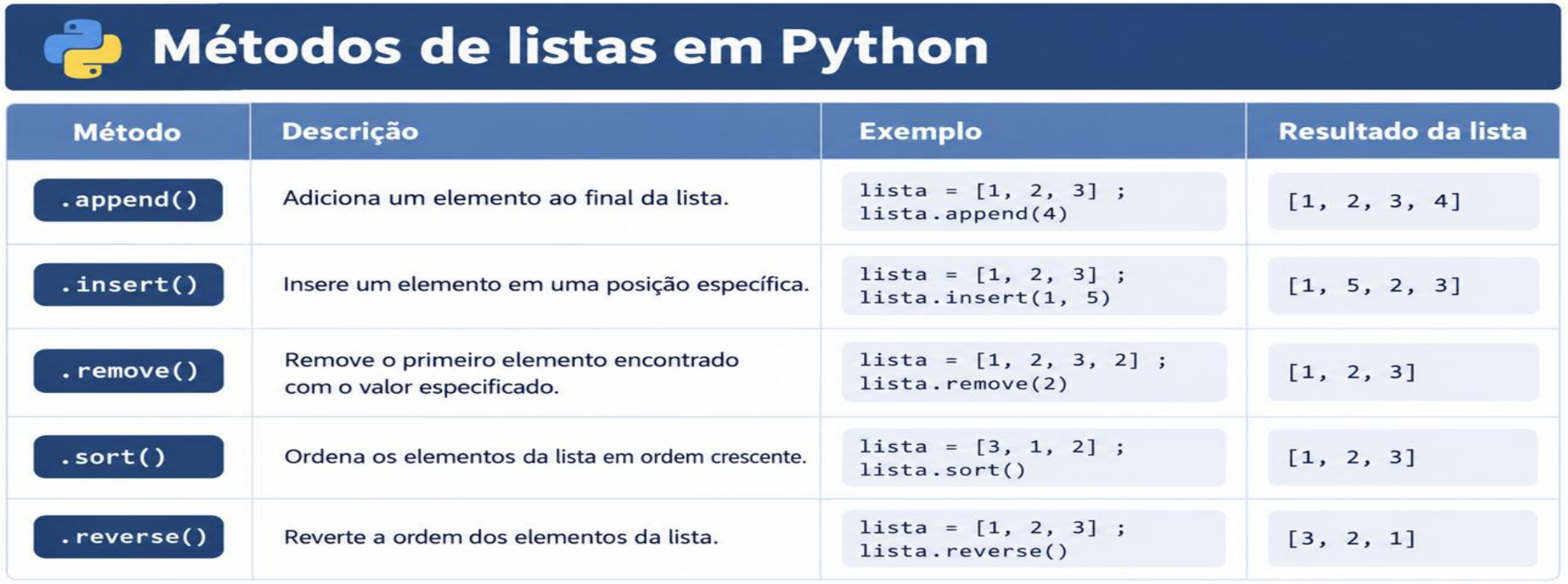
print(20 in lista) # Verifica se 20 está presente na lista, retorna True
print(40 in lista) # Verifica se 40 está presente na lista, retorna False
```

- No segundo print, quando verificamos se o 40 faz parte da lista, ele exibirá FALSE, porque o 40 não está na lista. O mesmo se aplica para as tuplas.

```
tupla = (10, 20, 30)
print(20 in tupla) # Verifica se 20 está presente na tupla, retorna True
print(40 in tupla) # Verifica se 40 está presente na tupla, retorna False
```

Manipulação de listas

- Além dessas operações comuns para listas e tuplas, temos alguns métodos específicos para listas, já que seus elementos podem ser alterados.



Métodos de listas em Python

Método	Descrição	Exemplo	Resultado da lista
`.append()`	Adiciona um elemento ao final da lista.	`lista = [1, 2, 3] ; lista.append(4)`	`[1, 2, 3, 4]`
`.insert()`	Insere um elemento em uma posição específica.	`lista = [1, 2, 3] ; lista.insert(1, 5)`	`[1, 5, 2, 3]`
`.remove()`	Remove o primeiro elemento encontrado com o valor especificado.	`lista = [1, 2, 3, 2] ; lista.remove(2)`	`[1, 2, 3]`
`.sort()`	Ordena os elementos da lista em ordem crescente.	`lista = [3, 1, 2] ; lista.sort()`	`[1, 2, 3]`
`.reverse()`	Reverte a ordem dos elementos da lista.	`lista = [1, 2, 3] ; lista.reverse()`	`[3, 2, 1]`

- O primeiro método é o append(), que adiciona um elemento no final da lista. No exemplo de código, temos uma lista com três elementos, chamamos a lista com o ponto append() e o valor que queremos adicionar ao final da lista. Ao fim dessa operação, a lista conterá quatro elementos.

Temos também na tabela os seguintes métodos:

- insert(): insere um elemento em uma posição específica dentro da lista.
- remove(): remove o primeiro elemento que encontrar com o valor especificado.
- sort(): ordena os elementos da lista em ordem crescente.
- reverse(): reverte a ordem dos elementos da lista.

Método pop()

- Além desses métodos, temos o método pop(), que, além de aplicar uma ação sobre a lista, retorna um valor.

```
lista = [1, 2, 3]
print(lista.pop(1))
print(lista)
```

No código, temos uma lista com três elementos e chamamos o pop() para a lista com o valor 1.

Esse valor é o índice do elemento que queremos remover da lista. Ele procura o elemento com esse índice, que é o valor 2, remove esse elemento e o retorna. O print trará na tela o valor 2. Por fim, imprimimos a lista atualizada com os valores 1 e 3.

Operações com tuplas

- Vimos como manipular listas, vamos ver que tipo de operações podemos fazer com as tuplas.
- Apesar das tuplas serem imutáveis, conseguimos criar novas tuplas a partir de tuplas existentes, através da concatenação de tuplas.

```
tupla1 = (1, 2)
tupla2 = (3, 4)
nova_tupla = tupla1 + tupla2
# A tupla nova_tupla contém todos os elementos de tupla1 seguidos pelos elementos de tupla2, resultando em (1, 2, 3, 4).
```

No código, temos a tupla 1, com os valores 1 e 2, e a tupla 2, com os valores 3 e 4. Criamos uma nova tupla através da concatenação da tupla 1 e da tupla 2, utilizando o operador +. Assim, a nova tupla resultará nos valores 1, 2, 3 e 4.

Iteração sobre elementos

- Outra ação que podemos fazer tanto com tuplas quanto com listas é percorrer todos os elementos para verificar quais são.
- Podemos iterar sobre esses itens utilizando laços de repetição, como laços for e while.

```
lista = [1, 2, 3, 4]
for item in lista:
    print(item)
tupla = (1, 2, 3, 4)
for item in tupla:
    print(item)
```

No código, temos uma lista com quatro elementos e chamamos um laço de repetição for, onde temos uma variável item, utilizamos o operador in e chamamos a lista. O laço percorrerá cada um dos elementos da lista e jogará esses valores para dentro da variável item. Dentro do laço, imprimimos o valor de cada um desses itens. Assim, conseguimos acessar todos os elementos de uma lista ou tupla um a um.

Desempacotamento

- Também conseguimos pegar os valores de uma tupla ou lista e atribuir esses valores a variáveis específicas, uma variável para cada valor, e isso é chamado de desempacotamento.

```
tupla = (1, 2, 3)
# Desempacota a tupla em 3 variáveis: a, b, c.
a, b, c = tupla
print(a, b, c)

lista = [10, 20, 30]
# Desempacota a lista em 3 variáveis: x, y, z.
x, y, z = lista
print(x, y, z)
```

- Para entender melhor, podemos verificar o código. Temos uma tupla com os valores 1, 2 e 3. Em seguida, criamos três variáveis: a, b e c, e atribuímos a tupla a essas variáveis. O código pegará cada um dos valores e os atribuirá às variáveis. Assim, a receberá o valor 1, b receberá o valor 2 e c o valor 3.
- Por fim, imprimimos os valores dessas variáveis, e isso se aplica também para as listas.

Onde utilizar listas e tuplas

- Agora, vamos entender onde podemos aplicar listas ou tuplas e qual é o momento ideal para utilizar cada uma dessas coleções.
- No primeiro exemplo, queremos criar um código que guardará uma lista de tarefas para o dia.
- Podemos adicionar itens, remover itens e, no final, imprimir a lista de tarefas atualizadas.
- Como a lista tem itens que podemos alterar, queremos excluir algumas tarefas, adicionar novas tarefas, o ideal é utilizar uma lista.

```
tarefas = ["Estudar Python", "Fazer compras", "Lavar a louça"]

tarefas.append("Ler um livro")  # Adiciona "Ler um livro" à lista
tarefas.remove("Fazer compras")  # Remove "Fazer compras" da lista

for tarefa in tarefas:
    print(tarefa)  # Exibe cada tarefa na lista
```

- No código, criamos uma lista chamada tarefas, com as tarefas de estudar Python, fazer compras e lavar louça. Em seguida, adicionamos uma nova tarefa na lista, chamando o método append().
- Em seguida, removemos a tarefa de fazer compras. Por fim, utilizamos um laço de repetição for para percorrer toda a lista, exibindo na tela as tarefas que ainda restam na lista.
- Agora, temos um exemplo com tuplas.
- Vamos criar um código onde armazenamos o nome de uma cidade e suas coordenadas de latitude e longitude. Em seguida, acessamos esses valores. Como o nome da cidade e as coordenadas são valores que não serão alterados, são fixos, o ideal é utilizar tuplas, por serem imutáveis.

```
# Definição de tuplas com nome da cidade, latitude e longitude
cidade1 = ("São Paulo", -23.5505, -46.6333)
cidade2 = ("Rio de Janeiro", -22.9068, -43.1729)
cidade3 = ("Brasília", -15.7801, -47.9292)

# Lista contendo as tuplas das cidades
cidades = [cidade1, cidade2, cidade3]

# Percorre a lista de cidades e desempacota os valores
for cidade in cidades:
    nome, latitude, longitude = cidade  # Atribui cada valor da tupla às variáveis
    print(f"Cidade: {nome}, Latitude: {latitude}, Longitude: {longitude}")  # Exibe as informações formatadas
```

- No código, criamos três variáveis para as cidades: São Paulo, Rio de Janeiro e Brasília, com suas respectivas latitudes e longitudes. Em seguida, pegamos cada uma dessas tuplas criadas com as informações das cidades e criamos uma lista.
- Por fim, temos um laço de repetição for que percorre essa lista e faz o desempacotamento dos valores das tuplas. Ele armazena as informações da cidade, o nome, a latitude e a longitude, dentro de variáveis.
- Para concluir, imprimimos esses valores na tela para cada uma das cidades.

FIM